Cosmos EVM IBC 통합 및 크로스체인 기능

1. IBC 통합 개요

1.1 IBC (Inter-Blockchain Communication) 소개

IBC는 서로 다른 블록체인 간의 안전한 통신을 가능하게 하는 프로토콜입니다.

1.2 Cosmos EVM에서의 IBC 활용

- **토큰 전송**: ERC20 토큰의 크로스체인 전송
- **스마트 컨트랙트 통신**: 체인 간 컨트랙트 호출
- **데이터 동기화**: 체인 간 상태 정보 공유

1.3 아키텍처 구조

```
┌─────────────────┐      IBC     ┌─────────────────┐
│     Chain A     │ ←──────────→ │     Chain B     │
│  (Cosmos EVM)   │              │  (Other Chain)  │
│                 │              │                 │
│ ┌─────────────┐ │                  │ ┌─────────────┐ │
│ │ ERC20 Token │ │                  │ │ Native Coin │ │
│ └─────────────┘ │                  │ └─────────────┘ │
└─────────────────┘                  └─────────────────┘
```

2. IBC Transfer 모듈 확장

2.1 기본 IBC Transfer vs Cosmos EVM Transfer

```
// 기본 IBC Transfer
type Transfer struct {
SourcePort  string
SourceChannel  string
Token sdk.Coin
Sender  string
Receiver  string
}

// Cosmos EVM 확장 Transfer
type EVMTransfer struct {
Transfer
// ERC20 토큰 지원
IsERC20  bool
ContractAddr  string
// 자동 변환 설정
AutoConvert  bool
}
```

2.2 Transfer Keeper 확장

```go
type Keeper struct {
// 기본 IBC Transfer 기능
*transferkeeper.Keeper
// ERC20 통합을 위한 추가 키퍼
erc20Keeper erc20keeper.Keeper
evmKeeper evmkeeper.Keeper
bankKeeper bankkeeper.Keeper
}
```

2.3 패킷 데이터 구조

```go
type FungibleTokenPacketData struct {
// 표준 IBC 필드
Denom  string  `json:"denom"`
Amount  string  `json:"amount"`
Sender  string  `json:"sender"`
Receiver  string  `json:"receiver"`
// Cosmos EVM 확장 필드
Memo  string  `json:"memo,omitempty"`
}
```

3. ERC20 IBC 미들웨어

3.1 미들웨어 스택 구조

```
┌─────────────────────────────────┐
| IBC Callbacks Middleware | ← 스마트 컨트랙트 콜백
├─────────────────────────────────┤
| ERC20 Middleware | ← ERC20 ↔ Cosmos 변환
├─────────────────────────────────┤
| IBC Transfer Module | ← 기본 IBC 전송
└─────────────────────────────────┘
```

3.2 ERC20 미들웨어 구현

```go
type IBCMiddleware struct {
app porttypes.IBCModule
keeper keeper.Keeper
}

func (im IBCMiddleware) OnRecvPacket(
ctx sdk.Context,
packet channeltypes.Packet,
relayer sdk.AccAddress,
) ibcexported.Acknowledgement {
// 패킷 데이터 파싱
```

```
var data transfertypes.FungibleTokenPacketData
if err := transfertypes.ModuleCdc.UnmarshalJSON(packet.GetData(), &data); err
return channeltypes.NewErrorAcknowledgement(err)
}
// ERC20 토큰 페어 확인
tokenPair, found := im.keeper.GetTokenPair(ctx, data.Denom)
if found && tokenPair.Enabled {
// 자동 ERC20 변환 수행
if err := im.keeper.ConvertCoinToERC20(ctx, data); err != nil {
return channeltypes.NewErrorAcknowledgement(err)
}
}
// 다음 미들웨어로 전달
return im.app.OnRecvPacket(ctx, packet, relayer)
}
```

3.3 송신 시 처리

```
func (im IBCMiddleware) OnSendPacket(
ctx sdk.Context,
packet channeltypes.Packet,
) error {
// ERC20 토큰을 Cosmos 코인으로 변환
```

```go
var data transfertypes.FungibleTokenPacketData
if err := transfertypes.ModuleCdc.UnmarshalJSON(packet.GetData(), &data); err
return err
}
// ERC20 → Cosmos 코인 변환
if err := im.keeper.ConvertERC20ToCoin(ctx, data); err != nil {
return err
}
return nil
}
```

4. 토큰 변환 메커니즘

4.1 송신 체인에서의 처리

```go
func (k Keeper) SendTransfer(
ctx sdk.Context,
sourcePort, sourceChannel string,
token sdk.Coin,
sender, receiver string,
) error {
// ERC20 토큰인지 확인
tokenPair, found := k.GetTokenPairByDenom(ctx, token.Denom)
```

```
if found {
// ERC20 토큰을 에스크로
if err := k.EscrowERC20Token(ctx, sender, token, tokenPair); err != nil {
return err
}
} else {
// 일반 Cosmos 코인 에스크로
if err := k.EscrowCoin(ctx, sender, token); err != nil {
return err
}
}
// IBC 패킷 전송
return k.transferKeeper.SendTransfer(ctx, sourcePort, sourceChannel, token, sen
}
```

4.2 수신 체인에서의 처리

```
func (k Keeper) OnRecvPacket(
ctx sdk.Context,
packet channeltypes.Packet,
) ibcexported.Acknowledgement {
var data transfertypes.FungibleTokenPacketData
if err := transfertypes.ModuleCdc.UnmarshalJSON(packet.GetData(), &data); err
```

```
return  channeltypes.NewErrorAcknowledgement(err)

}

// 토큰 페어 확인

tokenPair, found := k.GetTokenPair(ctx, data.Denom)

if  found && tokenPair.Enabled {

// ERC20 토큰으로 민팅

if  err := k.MintERC20Token(ctx, data.Receiver, data.Amount, tokenPair);  err !=

return  channeltypes.NewErrorAcknowledgement(err)

}

} else {

// 일반 Cosmos 코인으로 민팅

coin := sdk.NewCoin(data.Denom, sdk.NewIntFromString(data.Amount))

if  err := k.bankKeeper.MintCoins(ctx, transfertypes.ModuleName, sdk.NewCoins(co

return  channeltypes.NewErrorAcknowledgement(err)

}

// 수신자에게 전송

if  err := k.bankKeeper.SendCoinsFromModuleToAccount(ctx, transfertypes.ModuleNa

return  channeltypes.NewErrorAcknowledgement(err)

}

}

return  channeltypes.NewResultAcknowledgement([]byte{byte(1)})

}
```

5. IBC Callbacks 통합

5.1 Callbacks 미들웨어

```
type ContractKeeper interface {
// IBC 패킷 수신 시 컨트랙트 호출
IBCOnRecvPacket(
ctx sdk.Context,
packet channeltypes.Packet,
ack ibcexported.Acknowledgement,
) error
// IBC 패킷 확인 시 컨트랙트 호출
IBCOnAcknowledgementPacket(
ctx sdk.Context,
packet channeltypes.Packet,
acknowledgement []byte,
) error
// IBC 패킷 타임아웃 시 컨트랙트 호출
IBCOnTimeoutPacket(
ctx sdk.Context,
packet channeltypes.Packet,
) error
}
```

5.2 스마트 컨트랙트에서 IBC 콜백 처리

```
pragma solidity ^0.8.0;


interface IIBCCallback {
function onRecvPacket(
bytes calldata packet,
bytes calldata ack
) external;
function onAcknowledgementPacket(
bytes calldata packet,
bytes calldata acknowledgement
) external;
function onTimeoutPacket(
bytes calldata packet
) external;
}


contract CrossChainDEX is IIBCCallback {
function onRecvPacket(
bytes calldata packet,
bytes calldata ack
) external override {
```

```
// 크로스체인 토큰 수신 처리

// 자동으로 유동성 풀에 추가하거나 스왑 실행

}

function onAcknowledgementPacket(

bytes calldata packet,

bytes calldata acknowledgement

) external override {

// 전송 확인 처리

// 사용자에게 전송 완료 알림

}

}
```

6. 크로스체인 사용 사례

6.1 크로스체인 DEX

```
contract CrossChainDEX {

mapping(string => uint256) public liquidityPools;

function swapCrossChain(

string memory targetChain,

string memory tokenIn,

string memory tokenOut,

uint256 amountIn
```

```
) external {

// 1. 입력 토큰을 에스크로

IERC20(tokenIn).transferFrom(msg.sender, address(this), amountIn);

// 2. IBC를 통해 대상 체인으로 전송

bytes memory packetData = abi.encode(

msg.sender,

tokenIn,

tokenOut,

amountIn

);

// IBC 전송 실행 (프리컴파일 사용)

ICS20_PRECOMPILE.transfer(

targetChain,

"transfer",

packetData

);

}

}
```

6.2 크로스체인 스테이킹

```
contract CrossChainStaking {

struct StakeInfo {
```

```
uint256 amount;

string validatorAddr;

string sourceChain;

}

mapping(address => StakeInfo[]) public stakes;

function stakeCrossChain(

string memory targetChain,

string memory validatorAddr,

uint256 amount

) external {

// 1. 스테이킹할 토큰을 IBC로 전송

// 2. 대상 체인에서 자동으로 스테이킹 실행

// 3. 스테이킹 정보를 로컬에 저장

stakes[msg.sender].push(StakeInfo({

amount: amount,

validatorAddr: validatorAddr,

sourceChain: targetChain

}));

}

}
```

7. 보안 고려사항

7.1 토큰 검증

- **화이트리스트**: 허용된 토큰만 IBC 전송 가능
- **한도 설정**: 일일/월별 전송 한도 설정
- **다중 서명**: 대량 전송 시 다중 서명 요구

7.2 릴레이어 보안

- **릴레이어 검증**: 신뢰할 수 있는 릴레이어만 사용
- **타임아웃 설정**: 적절한 패킷 타임아웃 설정
- **재시도 로직**: 실패한 전송의 자동 재시도

7.3 스마트 컨트랙트 보안

- **재진입 공격 방지**: 콜백 함수에서 재진입 방지
- **가스 한도**: 콜백 실행 시 가스 한도 설정
- **권한 관리**: 콜백 함수 호출 권한 제한

이 문서는 Cosmos EVM의 IBC 통합과 크로스체인 기능을 상세히 설명합니다. 다음 문서에서는 JSON-RPC 서버와 이더리움 호환성을 살펴보겠습니다.